PRE-UNIVERSITY 2
COMPUTER SCIENCE
Standard and Higher Level

10 September 2026

75 minutes

Instructions to Candidates

- Answer all questions in the space provided directly below each question.
- For coding questions, write your code in the space provided. Do not modify any locked code shown.
- Show full working for evaluate/explain questions — marks are awarded for justified points, not just command words.
- The number of marks for each question is shown in [brackets] and gives an indication of the depth of answer required.

Name: _____ **Date:** _____ **Marks:** _____**/ 54**

Summative Assessment 1.1

Section A: Machine Learning

A1. Define the term machine learning. [2]

A2. Describe the key differences between supervised and unsupervised learning. [3]

A3. Evaluate the use of reinforcement learning in robotics navigation. [4]

A4. Outline what is meant by a field programmable gate array (FPGA). [1]

A5. A group of students would like to train and teach a machine learning model for a computer science school assignment on one of the students' laptops. They want this model to be able to recognize their classmates by an image given, based on the dataset that they have and will be providing.

a) Outline two hardware limitations in training a model with reference to the scenario above. [4]

b) Describe how supervised learning could be used in the above scenario. [4]

Section B: Object-Oriented Programming

Scenario: A car rental company is developing a software system to manage its fleet of vehicles. The system needs to represent different types of vehicles available for rent (e.g. cars, motorcycles, trucks) and calculate rental costs for customers. Some questions in this section use this scenario; others use a separate short scenario as indicated.

B1. Define the terms class and object in the context of object-oriented programming. [2]

B2. A UML class diagram is divided into three compartments (rows). State what information is contained in each compartment. [3]

B3. The rental company uses a Vehicle class to represent vehicles for rent. Distinguish between the Vehicle class and a Vehicle object such as car1, using an example from the scenario. [3]

B4. Evaluate the use of object-oriented programming for developing the vehicle rental management system described above. [4]

B5. Explain, using an example, how inheritance could be used to model different vehicle types (e.g. Car, Motorcycle, Truck) within this system. [3]

B6. Explain how polymorphism could be applied within this vehicle hierarchy, referring to a method such as calculate_rental_cost(). [3]

B7. [Coding Task — Encapsulation] [6 marks]

New scenario: A local library wants a simple system to track how many copies of a book are available to borrow.

Complete the LibraryBook class below so that it:

1. Stores title, total_copies, and available_copies as private attributes.

2. Has a borrow_copy() method that reduces available_copies by 1 — but only if at least one copy is available. If no copies are available, it should print "No copies available." and make no change.

3. Has a return_copy() method that increases available_copies by 1 — but available_copies must never exceed total_copies.

4. Has a public accessor method get_available_copies() that returns the current number of available copies.

Do not modify the driver code at the bottom of the cell — your class must work with it exactly as written.

```
class LibraryBook:
    def __init__(self, title, total_copies):
        # TODO 1: store title, total_copies, and available_copies as private
attributes
        pass
    def borrow_copy(self):
        # TODO 2: reduce available_copies by 1 if possible, otherwise print "No
copies available."
        pass
    def return_copy(self):
        # TODO 3: increase available_copies by 1, without exceeding total_copies
        pass
    def get_available_copies(self):
        # TODO 4: return the current number of available copies
        pass


# ----- DO NOT MODIFY BELOW THIS LINE -----
book1 = LibraryBook("Python Basics", 2)

print(book1.get_available_copies())   # Expected: 2
book1.borrow_copy()
print(book1.get_available_copies())   # Expected: 1
book1.borrow_copy()
print(book1.get_available_copies())   # Expected: 0
book1.borrow_copy()                   # Expected: No copies available.
print(book1.get_available_copies())   # Expected: 0
book1.return_copy()
book1.return_copy()
book1.return_copy()                   # should NOT exceed total_copies
print(book1.get_available_copies())   # Expected: 2
```

B8. [Extended Design Question] [6 marks]

The rental company now wants to add electric scooters to their system. Each electric scooter has a maximum battery range (in km) and a maximum speed, in addition to the attributes already shared by all vehicles (such as registration_number).

a) Justify whether ElectricScooter should be designed to inherit from the existing Vehicle class, referring explicitly to the "IS A" relationship test. [2]

b) The existing system contains code that loops through every vehicle in the fleet and calls vehicle.calculate_rental_cost() to generate a customer's bill, regardless of vehicle type. Explain how polymorphism would allow this existing code to correctly handle ElectricScooter objects without requiring any changes to that loop. [2]

c) Discuss one risk to the system if ElectricScooter were instead designed with a poor or forced inheritance relationship (for example, inheriting from an unrelated class purely to reuse code), and suggest one way this risk could be avoided. [2]

B9. [Coding Task — Polymorphism] [6 marks]

New scenario: An app needs to send notifications to users through different channels.

Complete the EmailNotification, SMSNotification, and PushNotification classes below so that each correctly overrides the send() method inherited from the Notification base class:

- EmailNotification.send() should return f"Sending email to {self.email_address}"
- SMSNotification.send() should return f"Sending SMS to {self.phone_number}"
- PushNotification.send() should return f"Sending push notification to device {self.device_id}"

Do not modify the Notification base class or the driver code at the bottom of the cell. Your classes must work with the driver code exactly as written, demonstrating that the same method call (send()) produces different, correct results depending on the object's actual class.

```
class Notification:
    def send(self):
        return "Sending generic notification."


class EmailNotification(Notification):
    def __init__(self, email_address):
        # TODO
        pass

    def send(self):
        # TODO
        pass


class SMSNotification(Notification):
    def __init__(self, phone_number):
        # TODO
        pass

    def send(self):
        # TODO
        pass


class PushNotification(Notification):
    def __init__(self, device_id):
        # TODO
        pass

    def send(self):
        # TODO
        pass


# ----- DO NOT MODIFY BELOW THIS LINE -----
notifications = [
    EmailNotification("user@example.com"),
    SMSNotification("555-1234"),
    PushNotification("DEVICE99")
]

for notification in notifications:
    print(notification.send())

# Expected output:
# Sending email to user@example.com
# Sending SMS to 555-1234
# Sending push notification to device DEVICE99
```